

UNIVERSITY OF VICTORIA
Department of Electrical and Computer Engineering
ECE 503 Optimization for Machine Learning

LABORATORY REPORT

Experiment No: *Lab 4*

Title: *Handwritten Digits Recognition Using Softmax Regression*

Date of Experiment: *Nov.17st, 2025*

Report Submitted on: *Nov.26th, 2025*

To: *Ruilin Wang*

Names: *Da Zhang (V01062902)*

1. Objective

The goal of this lab experiment is to build and employ a Softmax regression/classification model (with the regularized multi-class Softmax Cost Function) to distinguish the hand writing digits using a portion of the MNIST dataset. 16,000 training data are taken from the original MNIST data, which contains 60,000 samples. Specifically, 1600 randomly selected training samples from MNIST for each digit, 0 to 9. Also, 10,000 original MNIST testing data are used for test purposes. Each sample represents an image with 28*28 pixels and is characterized by 784 pixel information features. The objective is to classify each image sample as one of the digits from 0 to 9.

2. Introduction

The method we employed in this experiment is a multi-class classification model (with regularized multi-class Softmax Cost Function), where the data is split into training set with 16,000 samples and a training set with the 10,000 samples, and each of them contains 784 raw features. Besides, Histogram of oriented gradient (HOG) feature training and testing sets are also prepared with the same sample sizes but contains only 577 HOG features, which greatly reduces complexity/dimensionality. The optimal augmented weight parameters for the 784 original features and 1 bias term, and for the 577 HOG features and 1 bias term are computed based on minimizing the multi-class Softmax Cost Function, $E(\hat{w}_1, \hat{w}_2, \dots, \hat{w}_K) = \frac{1}{P} \sum_{p=1}^P \log \left(\sum_{j=1}^K e^{\hat{w}_j^T \hat{x}_p} \right) - \sum_{j=1}^K (\hat{w}_j^T \cdot \bar{x}_j) + \frac{\mu}{2} \sum_{j=1}^K \|\hat{w}_j\|_2^2$ on training sets to acquire the optimal classifier, $j^* = \arg \left(\max_{1 \leq j \leq K} \hat{w}_j^{*T} \hat{x} \right)$, using memoryless BFGS algorithm with BTLS technique for finding the step sizes. After that, the optimal classifier with the optimal augmented weight parameters are applied on the raw and HOG test sets to classify the testing samples. Lastly, the models' performances are evaluated in terms of confusion matrices and accuracy for both raw and HOG testing sets.

3. Implementation and Results

3.1. Implementation

1. Load raw digits and label matrices plus add the Lab_4 folder to the MATLAB path so helper functions are available.
2. Build the raw training/testing matrices: create a label row ytr, append it to X1600 for Dtr, and append labels to Te28 for Dte.
3. Generate HOG descriptors for each training image to form Dhtr, and repeat for the test images to form Dhtr with labels appended.

4. Add the SRMCC toolbox folder to the path and call SRMCC_bfgsML with Dtr, cost/gradient handles, and the recommended mu, K and iteration count to obtain classifier weights Ws.
5. Replace the label row of Dte with ones to create Dtest, compute class scores as Dtest' * Ws, and use max to get predicted class indices.
6. Populate a 10*10 confusion matrix C_te by counting how often each predicted/ground-truth label pair occurs, then display the matrix and compute accuracy via trace(C_te)/sum(C_te(:)).
7. Repeat the SRMCC training on HOG features (Dhtr) to get Ws_hog, construct Dhtest from Dhtr, predict labels, accumulate the HOG-based confusion matrix, and compute its accuracy.

3.2. MATLAB code

```
%% 5.1 Download and load data matrices
addpath('/Users/adamcheung/Documents/Uvic/Code/ECE503/Lab_4');
load /Users/adamcheung/Documents/Uvic/Code/ECE503/Lab_4/X1600.mat
load /Users/adamcheung/Documents/Uvic/Code/ECE503/Lab_4/Te28.mat
load /Users/adamcheung/Documents/Uvic/Code/ECE503/Lab_4/Lte28.mat

%% 5.2 Prepare two pairs of input data matrices, namely {Dtr, Dte} and {Dhtr, Dhtr}
Dtr = [X1600; ytr]; % Raw training data matrix with 784 raw features and 1 bias (785 * 16,000)
Dte = [Te28; 1+Lte28(:)']; % Raw testing data matrix with 784 raw features and 1 bias (785 * 10,000)

H = [];
for i = 1:16000
    xi = X1600(:,i);
    mi = reshape(xi,28,28);
    hi = hog20(mi,7,9);
    H = [H hi];
end

Hte = [];
for i = 1:length(Lte28)
    xi = Te28(:,i);
    mi = reshape(xi,28,28);
    hi = hog20(mi,7,9);
    Hte = [Hte hi];
end
```

```

Dhtr = [H; ytr]; % HOG training data matrix with 576 HOG features and 1 bias (577 * 16,000)
Dhte = [Hte; 1+Lte28(:)']; % HOG testing data matrix with 576 HOG features and 1 bias (577 * 10,000)

%% 5.3 Apply SRMCC to the original datasets to classify the test samples
addpath('/Users/adamcheung/Documents/Uvic/Code/ECE503/Lab_4/SRMCC');

[Ws, f] = SRMCC_bfgsML(Dtr, "f_SRMCC", "g_SRMCC", 0.002, 10, 62); % Recommended mu = 0.002, K = 10 and iter = 62;

% Predict the labels for test samples
Dtest = [Dte(1:end-1, :); ones(1, size(Dte, 2))]; % Add bias row to test data matrix
tic; % Measure recognition time for raw features
scores = Dtest' * Ws;
[~, ypred] = max(scores, [], 2);
raw_recognition_time = toc;

% Construct 10*10 confusion matrix
C_te = zeros(10,10);
ytrue = Dte(end, :);
for i = 1:numel(ypred)
    predIdx = ypred(i);
    gtIdx = ytrue(i);
    C_te(predIdx, gtIdx) = C_te(predIdx, gtIdx) + 1;
end
disp("Confusion Matrix for raw features testing dataset:")
disp(C_te);

% Compute prediction accuracy
accuracy_te = trace(C_te) / sum(C_te(:));
disp("Prediction Accuracy for raw features testing dataset:")
disp(accuracy_te);
raw_digits_per_second = numel(ypred) / raw_recognition_time;

% Return recognition speed (efficiency)
disp("Recognition speed (raw features) - digits per second:")
disp(raw_digits_per_second);

%% 5.4 Apply SRMCC to the HOG feature datasets to classify the test samples
addpath('/Users/adamcheung/Documents/Uvic/Code/ECE503/Lab_4/SRMCC');

[Ws_hog, f_hog] = SRMCC_bfgsML(Dhtr, "f_SRMCC", "g_SRMCC", 0.001, 10, 57); % Recommended mu = 0.001, K = 10 and iter = 57;

% Predict the labels for hog test samples

```

```

Dhtest = [Dhte(1:end-1, :); ones(1, size(Dhte, 2))]; % Add bias row to test data
matrix
tic; % Measure recognition time for HOG features
scores = Dhtest' * Ws_hog;
[~, ypred_hog] = max(scores, [], 2);
hog_recognition_time = toc;

% Construct 10*10 confusion matrix
C_te_hog = zeros(10,10);
ytrue = Dhte(end, :);
for i = 1:numel(ypred_hog)
    predIdx = ypred_hog(i);
    gtIdx = ytrue(i);
    C_te_hog(predIdx, gtIdx) = C_te_hog(predIdx, gtIdx) + 1;
end
disp("Confusion Matrix for HOG features testing dataset:")
disp(C_te_hog);

% Compute prediction accuracy
accuracy_te_hog = trace(C_te_hog) / sum(C_te_hog(:));
disp("Prediction Accuracy for HOG features testing dataset:")
disp(accuracy_te_hog);
hog_digits_per_second = numel(ypred_hog) / hog_recognition_time;

% Return recognition speed (efficiency)
disp("Recognition speed (HOG features) - digits per second:")
disp(hog_digits_per_second);

```

3.3 Results

Results for 5.3.1 Report the confusion matrix, accuracy and efficiency for original testing data

```

Confusion Matrix for raw features testing dataset:
961      0      10      2      1      9      9      3      6      12
0      1111      8      1      6      4      3      10      9      7
0      2      908      21      4      1      5      20      10      2
2      2      15      913      1      34      2      8      22      12
1      0      12      0      907      12      11      6      7      31
4      3      2      29      1      772      13      1      27      7
10      4      13      2      12      18      911      0      14      0
1      1      16      12      2      7      3      941      12      24
1      12      39      18      8      25      1      2      856      10
0      0      9      12      40      10      0      37      11      904

Prediction Accuracy for raw features testing dataset:
0.9184

Recognition speed (raw features) - digits per second:
2.8737e+06

```

Fig 1. Confusion Matrix, Classification Accuracy and Recognition Efficiency for classifier applied on raw testing data (model trained on raw data with $u = 0.002$, $K = 10$ and $\text{iter} = 62$)

Results for 5.4.1 Report the confusion matrix, accuracy and efficiency for HOG testing data

Confusion Matrix for HOG features testing dataset:										
974	3	4	0	1	2	3	0	3	6	
0	1128	6	0	2	1	3	4	1	4	
0	0	1009	0	1	1	0	10	3	1	
0	0	6	992	0	3	1	3	2	2	
0	0	0	0	964	0	3	1	2	9	
0	0	0	8	0	876	5	0	2	2	
3	1	1	0	2	1	939	0	2	0	
1	3	3	4	2	0	0	1001	2	7	
2	0	3	3	2	6	3	3	953	10	
0	0	0	3	8	2	1	6	4	968	
Prediction Accuracy for HOG features testing dataset:										
0.9804										
Recognition speed (HOG features) – digits per second:										
3.6489e+06										

Fig.2 Confusion Matrix, Classification Accuracy and Recognition Efficiency for classifier applied on HOG testing data (model trained on HOG data with $u = 0.001$, $K = 10$ and $\text{iter} = 57$)

Results for 4.6.2 Comment on the accuracy obtained and efficiency

The classifier trained using HOG features demonstrates a clear superiority in both accuracy and computational efficiency. While the raw-feature classifier reaches 91.3% accuracy, the HOG-based classifier achieves 98.0%, reflecting much better discrimination across digits as seen in its cleaner confusion matrix. Furthermore, the HOG model processes approximately 3.65 million digits image per second, outperforming the raw-feature model at 2.87 million digits per second. These result confirm that HOG provides a more compact and informative representation of digit shapes, enabling both faster and more accurate classification.

4. Discussion

A comparative evaluation of the two classifier trained using raw/original pixel features and HOG features demonstrates that the HOG-based classifier substantially outperforms its raw-feature counterpart in both predictive accuracy and computational efficiency. The classifier trained on raw pixel intensities achieves an accuracy of 91.84%, with its confusion matrix revealing notable misclassifications among visually similar digits such as 3/5, 4/9 and 5/8. In contrast, the HOG-feature classifier attains a significantly higher accuracy of 98.04%, exhibiting a markedly cleaner confusion matrix with far fewer off-diagonal errors. This improvement reflects the superior discriminative power of HOG descriptors, which capture edge orientation and local shape information more effectively than raw pixel values.

Additionally, the HOG-based model demonstrates faster recognition speeds, processing approximately 3.65 million digits images per second compared to 2.87 million for the raw-feature model, suggesting that the reduced dimensionality and enhanced feature structure of HOG descriptors lead to more efficient computation.

5. Conclusion

In this experiment, we successfully implemented a multi-class classification model with Regularized multi-class Softmax Cost Function to classify hand-written digits using memoryless BFGS algorithm with BTLS. Also, this study compared the performance of classifiers trained on raw pixel features versus HOG descriptors – The results consistently demonstrate that the HOG-based classifier is markedly superior, achieving higher accuracy (98.04% vs. 91.84%), fewer misclassifications, and greater recognition speed. The success of the HOG-based approach highlights the importance of feature engineering in machine learning, especially in tasks involving structured visual patterns. While raw pixels provide a direct but noisy and high-dimensional representation, HOG features offer a more meaningful abstraction that captures essential geometric characteristics of the digits, namely the magnitude of brightness change and the direction of brightness change. As a result, the HOG model not only generalizes better but also computes predictions more efficiently.

Overall, the comparison underscores that incorporating domain-appropriate feature extraction can significantly enhance both the effectiveness and scalability of classification systems, highlighting the importance of feature engineering. For such image recognition tasks, HOG-based approach provides a clear advantage and represents a more robust solution than using the raw feature information directly.

6. References

None.